Ejercicio RSA

Fecha de Entrega: 17 de marzo de 2026

Objetivos de Aprendizaje

Al completar este laboratorio, el estudiante será capaz de:

- Explicar la fundamentación matemática de RSA: factorización de primos, aritmética modular y Teorema de Euler.
- Generar pares de claves RSA de longitud adecuada **(2048/3072 bits)** con pycryptodome.
- Implementar cifrado híbrido **RSA-OAEP + AES-256-GCM** para datos de tamaño arbitrario.
- Identificar vulnerabilidades derivadas del uso incorrecto de padding: PKCS#1 v1.5 vs OAEP.
- Analizar la presencia de RSA en protocolos reales: TLS 1.3, certificados X.509, SSH.

Entregables:

- Repositorio Git con el código fuente completo
- README.md actualizado que incluya:
  - Descripción del proyecto
  - Instrucciones de instalación y uso
  - Ejemplos de ejecución
  - Respuestas a las preguntas de análisis
- Implementación: Código en el lenguaje de su preferencia (puede usar Jupyter Notebook si elige Python)

Recursos Recomendados

- Documentación del módulo random o secrets en Python
- Presentaciones de clase https://locano-uvg.github.io/cifrados-26/
- pycryptodome: https://pycryptodome.readthedocs.io
- RFC 8017 — PKCS#1 v2.2 (OAEP): https://www.rfc-editor.org/rfc/rfc8017

Escenario:

Plataforma de Transferencia de Documentos Legales

Una firma de abogados necesita transferir documentos confidenciales entre sus oficinas de Guatemala City, Miami y Madrid. Los documentos contienen contratos, acuerdos de confidencialidad y datos personales.

El sistema debe garantizar:

- Solo el destinatario pueda leer el documento

Problemas:

1. ***El sistema usa RSA como mecanismo de intercambio de clave, protegiendo una clave AES que cifra el documento real.***
   a. ¿Explique por qué no cifrar el documento directamente con RSA?

2. **Generación de Claves**

Implementa la generación de claves RSA en el archivo generar_claves.py

- Genera un par de claves RSA de bits.
- Exporta ambas claves en formato PEM.
- Guardar:
  - private_key.pem (protegida con passphrase "lab04uvg")
  - public_key.pem

```python
from Crypto.PublicKey import RSA

def generar_par_claves(bits: int = 3072):
    # TODO: implementar con RSA.generate()
    # Recordar: mínimo 2048 bits; recomendado 3072 para nuevo software
    pass

if __name__ == '__main__':
    generar_par_claves(3072)
    print("Claves generadas: private_key.pem y public_key.pem")
```

Después de implementar, respondan:

- ¿Qué información contiene un archivo .pem? Abre public_key.pem con un editor de texto y describe su estructura.

3. Cifrado y Descifrado Directo con RSA-OAEP

a. Cifra 'mensaje' usando RSA-OAEP
b. Decifrar usando la clave privada RSA

```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

def cifrar_con_rsa(mensaje: bytes, public_key_pem: bytes) -> bytes:
    # TODO: implementar
    pass

def descifrar_con_rsa(cifrado: bytes, private_key_pem: bytes) -> bytes:
    # TODO: implementar
    pass
```

Después de implementar, respondan:

- ¿Porqué cifrar el mismo mensaje dos veces produce resultados distintos? Demuéstrenlo y expliquen que propiedad de OAEP lo cause

4. Cifrado Híbrido RSA-OAEP + AES-GCM

a. Generar clave AES aleatoria de 256 bits
b. Cifrar el documento con AES-GCM
    i. El resultado les dará nonce + tag + ciphertext
c. Cifrar la clave AES con la clave pública RSA usando PKCS1_OAEP

```python
import os
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from generar_claves import generar_par_claves


def encrypt_document(document: bytes, recipient_public_key_pem: bytes) -> bytes:
    # 1. Cifrar el documento con AES-256-GCM
    # 2. Cifrar la clave AES con la clave pública RSA (OAEP)

    return True

def decrypt_document(pkg: bytes, recipient_private_key_pem: bytes) -> bytes:
    return True

if __name__ == '__main__':
    generar_par_claves(2048)

    with open("public_key.pem", "rb") as f: pub = f.read()
    with open("private_key.pem", "rb") as f: priv = f.read()

    # Generen un cifrado de un texto
    doc = b"Contrato de confidencialidad No. 2025-GT-001"
    pkg = encrypt_document(doc, pub)
    resultado = decrypt_document(pkg, priv)


    # Prueba con archivo de 1 MB (simula un contrato real)
    doc_grande = os.urandom(1024 * 1024)
    pkg2 = encrypt_document(doc_grande, pub)
    assert decrypt_document(pkg2, priv) == doc_grande
    print("Archivo 1 MB: OK")
```